Task 1: Multi-Environment IAM with Least Privilege

GCP IAM Design & Implementation

Project: cicd-pipeline-demo-494502

1. Project Overview

This document describes the IAM architecture designed and deployed on Google Cloud Platform (GCP) for a development team requiring multi-environment access to Google Kubernetes Engine (GKE). The solution follows the least privilege principle using custom roles, service accounts with Workload Identity, and time-based conditions.

Team composition:

- 3 Developers — deploy access to dev environment, read-only to staging and prod
- 2 DevOps Engineers — full access to all environments
- 1 QA Engineer — view logs and metrics only across all environments
- 1 CI/CD Service Account — automated deployment to all environments

2. IAM Architecture

2.1 Custom Roles

Five custom roles were created instead of using predefined GCP roles. This ensures each identity receives only the exact permissions required for their function.

Role ID	Title	Purpose	Key Permissions
gke_developer	GKE Developer	Deploy to dev environment	deployments.create/update, pods.create/delete
gke_viewer	GKE Viewer	Read-only staging/prod	clusters.get, pods.get, services.get
devops_admin	DevOps Admin	Full GKE access all envs	clusters.create/delete, namespaces.create
qa_observer	QA Observer	Logs and metrics view only	logEntries.list, timeSeries.list
cicd_deployer	CICD Deployer	Automated deployments	deployments.create, artifactregistry.get

2.2 IAM Bindings with Conditions

IAM bindings assign custom roles to team members. A time-based condition restricts developer access to business hours only (9am-6pm UTC, Monday-Friday), reducing the attack window.

Member	Role Assigned	Environment	Condition
3 Developers	gke_developer	Dev only	Business hours (9am-6pm UTC)
3 Developers	gke_viewer	Staging + Prod	None (read-only always allowed)
2 DevOps Engineers	devops_admin	All environments	None (24/7 incident response)
1 QA Engineer	qa_observer	All environments	None
cicd-deployer-sa	cicd_deployer	All environments	None

3. Service Accounts & Workload Identity

3.1 Service Accounts Created

Account ID	Email	Purpose
cicd-deployer-sa	cicd-deployer-sa@cicd-pipeline-demo-494502.iam.gserviceaccount.com	CI/CD pipeline deployments
gke-workload-sa	gke-workload-sa@cicd-pipeline-demo-494502.iam.gserviceaccount.com	Workload Identity for GKE pods

3.2 Workload Identity Configuration

Workload Identity Federation binds the Kubernetes Service Account (gke-workload-ksa) in the default namespace to the GCP Service Account (gke-workload-sa). This eliminates the need for long-lived JSON key files, which is a significant security improvement.

Binding:

K8s SA: default/gke-workload-ksa → GSA: gke-workload-sa@cicd-pipeline-demo-494502.iam.gserviceaccount.com

Role granted: roles/iam.workloadIdentityUser

4. Design Decisions

Decision	Rationale
Custom roles over predefined roles	Predefined roles like roles/container.developer grant far more permissions than needed. Custom roles ensure exact least-privilege access.
Time-based condition for developers	Restricting developer deploy access to business hours (9am-6pm UTC) reduces the attack window significantly. Any compromised developer credential is useless outside working hours.
Workload Identity over service account keys	JSON key files are long-lived credentials that can be leaked or stolen. Workload Identity uses short-lived tokens automatically rotated by GCP, with zero key management overhead.
Separate CI/CD service account	Isolating the CI/CD identity allows granular audit logs, easy revocation without affecting human users, and a clear security boundary between human and automated access.
QA has zero write permissions	QA engineers observe system behavior but should never modify production. Giving them only logs and metrics view prevents accidental changes in any environment.
DevOps has no time-based conditions	DevOps engineers respond to incidents 24/7. Restricting their hours would prevent emergency responses, creating more risk than it mitigates.
IAM bindings at project level	For a single-project setup, project-level bindings are appropriate. In a multi-project organization, folder-level or org-level bindings with environment-specific labels would be used.

5. Terraform File Structure

The implementation is organized into 7 Terraform files for clear separation of concerns:

File	Contents
main.tf	Provider configuration (hashicorp/google ~> 5.0)
variables.tf	Input variable declarations for project, region, user emails
terraform.tfvars	Variable values: project ID, team email addresses
custom_roles.tf	5 custom IAM role definitions with granular permissions
service_accounts.tf	2 service accounts + Workload Identity binding
iam_bindings.tf	Role-to-member bindings with time-based conditions
outputs.tf	Output values: service account emails after apply